Arrays

- contiguous space in the memory
  - Eg: int[50] → 50×4 = 200 bytes
    - int takes 4 bytes
- @600 @604 @608

0 1 2 49

addr of element at index 'i' = @600 + (i × 4)

- @600 → base address
- 4 → size of each element

O(1) operation
or constant time
(doesn't depend on n)

Random access is very fast - arr[i]
or arr[i] = x;

- Disadvantages:
  - we have to know the size beforehand
  - size is fixed ⇒ have to create a new array & copy over old elements
  - insertion / deletion at beginning/middle - O(n)

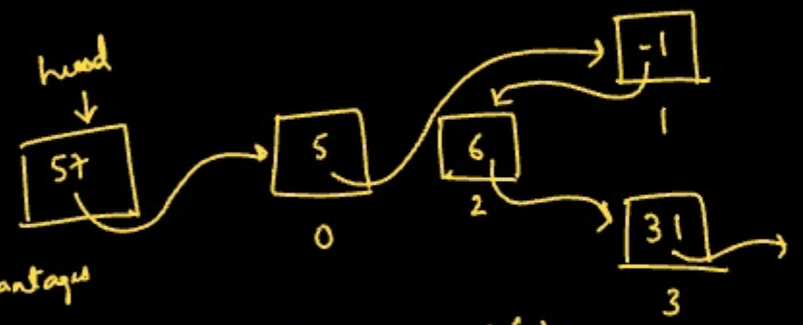
insert 8 at index '0'
- move all elements one step to the right & insert 8 at index 0.
  - O(n) operation

delete at index '0'
- delete element at index 0 & move all elements one step to left
  - O(n)

Advantages:
- Insertion/deletion at end - O(1)
- Random access - O(1)

Linked lists :

- not contiguous (all elements) in memory
-



advantages

insert at beginning - O(1)

delete at beginning - O(1)

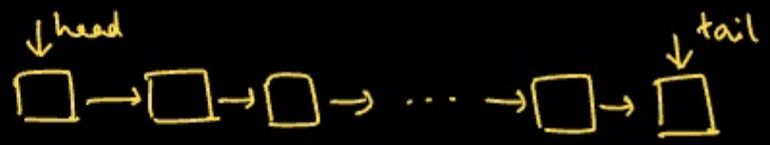
doesn't have to be fixed size

we don't need to know the number of elements before hand.

Disadvantages

- Random access now takes O(n) time
  - we need to start at the head & jump to the ith element.
- Insertion at end - O(n)
- Deletion at end - O(n)

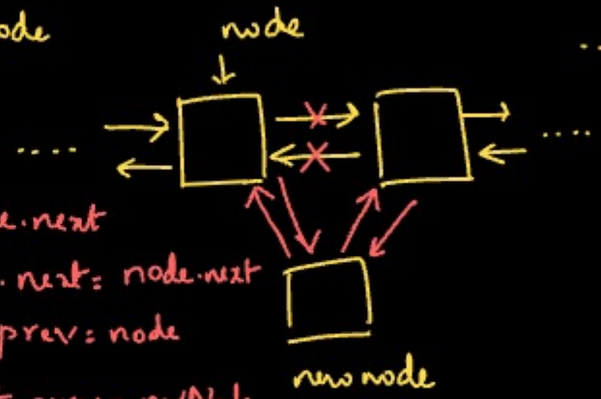
-



- If we have a tail stored
  - insertion at end - O(1)
  - deletion at end - O(n)

- Doubly linked lists
  - has previous & next pointers from each node

- Insertion & deletion at end - O(1)
- add new node after "node"



optional → next = node.next

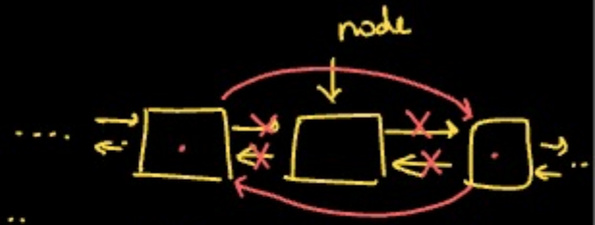
newnode.next = node.next

newnode.prev = node

node.next.prev = newNode

node.next = newNode

RT: O(1) local operation



- delete node from the list (assume the nodes before & after "node" are not null)

node.next.prev = node.prev

node.prev.next = node.next

node.next = null

node.prev = null

O(1) operation

- Induction
  - Students in a line
  - Each one wants a chocolate
  - If
    1. first student gets a chocolate
    2. If i^{th} student gets a chocolate, then $(i+1)^{th}$ student also gets one.

To Prove: $1+2+3+\dots+n = \frac{n(n+1)}{2} = S(n)$

1. Base case: $n=1$

   $S(1) = 1$

   $\frac{n(n+1)}{2} = \frac{1\times 2}{2} = 1$

2. Inductive step:

   To prove: Given $S(n) = \frac{n(n+1)}{2}$, that $S(n+1) = \frac{(n+1)(n+2)}{2}$

   TP: $S(n+1) = \frac{(n+1)(n+2)}{2}$

   $LHS = S(n+1)$

   $= S(n) + (n+1)$

   $= \frac{n(n+1)}{2} + (n+1) = (n+1)\left(\frac{n}{2}+1\right) = \frac{(n+1)(n+2)}{2}$ □

- Proof: $1\times 2 + 2\times 3 + \dots + n(n+1) = \frac{n(n+1)(n+2)}{3} = S(n)$

  Base case

  $n=1$

  $S(1) = 1\times 2 = 2$

  $\frac{1(1+1)(1+2)}{3} = 2$

  Inductive step

  Given $S(n) = \frac{n(n+1)(n+2)}{3}$

  To prove, $S(n+1) = \frac{(n+1)(n+2)(n+3)}{3}$

  $S(n+1) = \underbrace{S(n)}_{1\times 2 + 2\times 3 + \dots + n(n+1)} + (n+1)(n+2)$

  $= \frac{n(n+1)(n+2)}{3} + (n+1)(n+2)$

  $= (n+1)(n+2)\left(\frac{n}{3}+1\right)$

  $=$ RHS □

Recursion

- Write a fn that finds n! (iterative)

```
def factorial(n):
    fact = 1
    for i in range(1, n+1):
        fact *= i
    return fact
```

-
```
def factorial_rec(n):
    if n==0: return 1
    else:
        return n * factorial_rec(n-1)
```

0! = 1

↓

assuming that it works for n-1 & using it to compute for n.

- Recursion:
  1. Break the problem into a smaller problem of EXACTLY the same type.
  2. Assume it works on the smaller problem & use it to solve the actual problem.

- Eg: Write a recursive fn to test if a string is a palindrome.

```
def is_palindrome(s):
    #base case
    if len(s) <= 1: return True

    #recursive case
    middle = s[1:-1]

    return
    s[0] == s[-1] and is_palindrome(middle)
```

ABA ✓
ACCA ✓
ABCA ✗
A ✓
"" ✓
MADAM ✓

- Eg: Write a recursive fn to reverse a string

```
def reverse(s):
    if len(s) <= 1: return s

    # recursive case
    return s[-1] + reverse(s[1:-1])
                 + s[0]
```

rev("") → ""
rev(A) → A
rev(ABCD) → D + rev(ABC)
rev(BCD) + A
D + rev(mid) + A

Eg: Write a fn to find the sum of digits of a +ve number

1. iteratively
2. recursively

(1)

```
def sum_of_digits(n):
    sum = 0
    while n>0:
        sum += n % 10
        n = n // 10    #update
    return sum
```

1054 → 10
12345 → 15
1000 → 1

1054 % 10 → 4
1054 // 10 → 105

floor division
integer division

105 % 10 → 5
105 // 10 → 10

10 % 10 → 0
10 // 10 → 1

1 % 10 → 1
1 // 10 → 0

(2)

```
def sum_rec(n):
    #base case
    if n==0: return 0
    #recursive case
    return sum_rec(n//10) + (n%10)
```

sum(1054) → sum(105) + 4

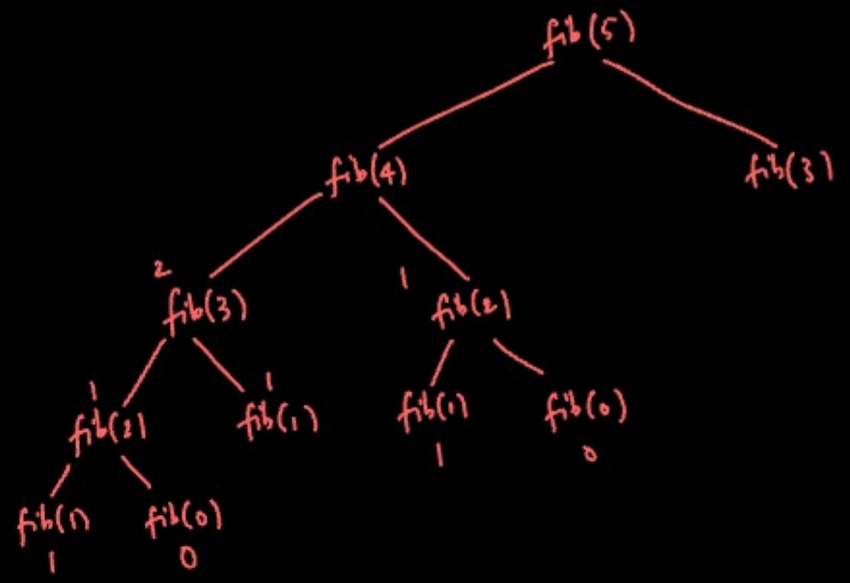
Eg: 0, 1, 1, 2, 3, 5, 8, 13, ...
(0 1 2 3 4 5 6 7 ...)

fib(n) = fib(n-1) + fib(n-2)

fib(0) = 0

fib(1) = 1

```
def fib(n):
    if n==0 : return 0
    if n==1 : return 1
    return fib(n-1) + fib(n-2)
```



fib calls is exponential : 2^n

n=50 2^{50} steps ~ 10^{17} steps → 10^8 sec ~3 years

1 GHz → 10^9 steps / sec

10^9 per second

```
memo = { }      #result of fib
def fib(n):
    if n in memo: return memo[n]
    if n==0 or n==1: return n

    result = fib(n-1) + fib(n-2)
    memo[n] = result
    return result
```

```
def fib(n):
    a = 0
    b = 1
    c = a+b
    a = b
    b = c
```

1 Billion'th fib -> $O(n)$ -> 10^9 steps => 1s to compute

Space complexity: Amt of <u>extra</u> space we use other than input & output

$O(n)$ for above problem

- is_palindrome recursive fn

```
def is_palindrome(s):
    if len(s) <= 1 : return True
    return  s[0] == s[-1]  and
                is_palindrome(s[1:-1])
```

```
def is_palindrome(arr_chars, i, j):
    if  i == j : return True
    return  s[i] == s[j]  and
                is_palindrome(arr_chars, i+1, j-1)
```

i & j are inclusive

```
is_palindrome( arr, 0, len(arr)-1)
```

↓

```
["M", "A", "D", "A", "M"]
 0                    4
```

```
def is_palindrome(arr, i, j) :
    # base case
    first = arr[i]
    mid_result = is_palindrome(arr, i+1, j-1)
    last : arr[j]
    return (first == last) and mid_result
```

- Immutability:
  - Once an object is created, it cannot be modified

```
arr = ['M', 'A', 'D', 'A', 'M']

is_Palindrome(0, 4):
    arr[0]
    arr[4]
    is_Palindrome(1, 3)
        arr[1]
        arr[3]
        is_palindrome(2,2):
            True
```

arr[0] → loads 0 to 6
arr[1]
arr[2]

```
f(0, n-1):
    arr[0]
    f(1, n-2):
        arr[1]
        f(2, n-3):
            arr[2]
            f(3, n-4) . . . { arr[3]
                              ⋮
                              arr[n-4]
            arr[n-3]
        arr[n-2]
    arr[n-1]
```

- Abstraction:

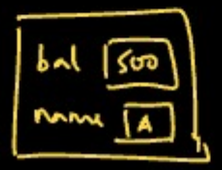
- Motivation:
  - C – no classes
    - 3 parallel arrays
      1. float[] for balances
      2. string[] for names
      3. int[] for ages

- classes: have the blueprint of what data (attributes) & what behaviour (methods) we need.

- objects: are the concrete implementations of classes
  - Multiple objs can be created from the same class.
  - Each obj has its own data & behaviour

```
class Account {
    double bal;
    String name;        } data

    void incrementBal (double x) {...}
}
```



Java classes

public
private
protected
default

```
public class Account {
    private double balance;
    private String name;

    public Account(double b, String n) {
        this.balance = b;
        this.name = n;
    }
    public double getBal() { return bal; }
    public void setBal(double b) {
        // perform checks
        // only set valid values
        this.bal = b;
    }
}
```

Constructor is defined like a fn in Java. No return type. Name of fn = Name of class

this points to the current object on which the method is called

```
Account a1 = new Account(500, "A");
p(a1.bal);      // 500
p(a1.name);     // A
a1.bal = -20000;    // not valid
```

⇒ Make attributes private & give controlled access using getters & setters

Memory Models

- Data types:
  - primitive types – int, char, float, double, byte, boolean...
    - starts with a lower case
    - stored on stack
  - Object type – they have their own classes
    - stored on heap.

static:

- public class A {

```
    private static int cnt = 0;

    public A() {
        ....
        cnt++;
    }
}
```

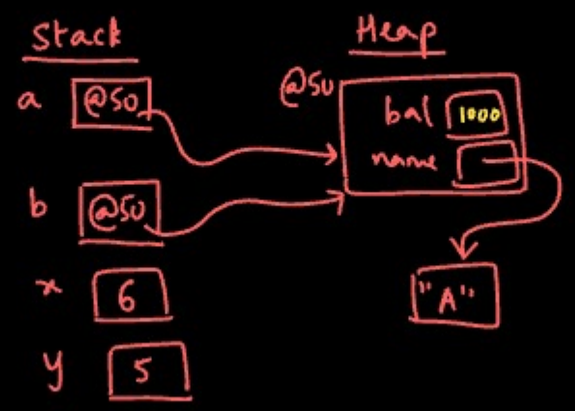
x++ & ++x

- int x = 5;
  int y = (++x)^2;
  int z = $\underbrace{(x++)}_{6} + \underbrace{(++y)}_{37}$;
  print(x, y, z);

x	7
y	37
z	43

- Memory models
  - Variables are different from actual objects

```
Account a;
a = new Account(500, "A");
Account b = a;   a.setBal(1000);
                 print(b.getBal());
                                  // 1000
int x = 5;
int y = x;
x++;
print(x); // 6
print(y); // 5
```

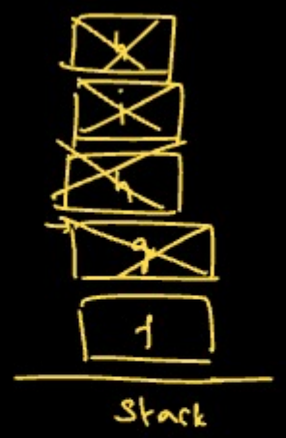


  - primitive types are stored on stack
  - object types are created on heap

int, double, char ...

all the rest

  - Variables are created on stack.



f()
g()
h()
i()
h()

LIFO

We destroy a frame of a fn once it is done running.

To preserve things across function calls, use heap

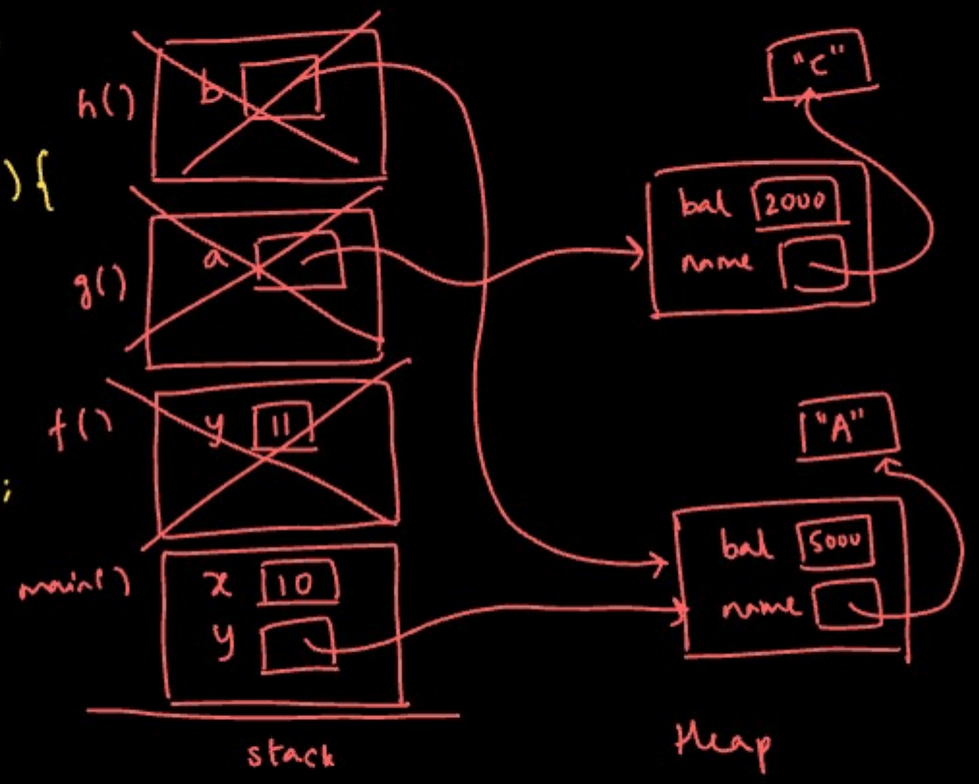
```
private static void f(int y) {
        y++;
}

private static void g(Account a) {
        a = new Account(2000, "C");
}

private static void h(Account b) {
        b.setBal(5000);
}

main() {
    int x = 10;
    Account y = new Account(1000, "A");
    f(x);
    print(x); // 10
    g(y);
    print(y.getBal()); // 1000
    h(y)
    print(y.getBal()); //5000
}
```

Entry point



```java
public class SimpleLocation
{
    public double lat;
    public double lon;

    public SimpleLocation(
        double latIn,
        double lonIn)
    {
        this.lat = latIn;
        this.lon = lonIn;
    }
    // More code here
}
```

```java
public class LocationTester
{
  public static void main(String[] args)
  {
      SimpleLocation loc1 =
        new SimpleLocation(39.9, 116.4);
      SimpleLocation loc2 = ;
        new SimpleLocation(55.8, 37.6);
      loc1 = loc2;
      loc1.lat = -8.3;
      System.out.println(loc2.lat +
                         ", " +loc2.lon);
  }
}
```

- Access Modifiers
  - public → accessible from everywhere
  - private → accessible from within the class
  - protected → from class, from subclasses, from the same package
  - default → from class, from the same package

dsa/
  sorting/
    Merge.java
    Insertion.java
    SortUtils.java

```java
public class Student {
    String name;
    int age;

    public Student(String name, int a) {
        this.name = name;
        age = a;
    }
    public void setAge(int age) {
        this.age++;
        age++;
    }
}
```

```java
int x=5;
x=x;
```

- function signatures
  - name of the function + parameters of the function form the signature
  - access modifiers & return type are not part of the signature
  - int sum(int a, int b)
    int sum(int x, int y) > same signature
  - int sum(int x, int y)
    double sum(int x, int y) > same signature

- Method overloading
  - We can have multiple functions with the same name as long as the signature is different.
  - Especially useful in recursive fns & wrapper fns & also constructor overloading

void sort (int[] array)
↓
void sort (int[] array, int lo, int hi)

```
int sum(int x, int y){

}
```

x, y → parameters of the fn

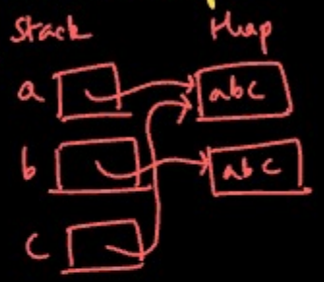
values we pass for x & y while calling the fn → arguments

- equals() vs == → checks if they are the same object
  - we can have a custom implementation of how two objects should be equal.

```
String a = new String("abc");
String b = new String("abc");
String c = a;

a == b; // false
b == c; // false
c == a; // true

a.equals(b); // true
b.equals(c); // true
c.equals(a); // true
```



- String Interning
  - Optimization
  - String a = "abc";
    String b = "abc";

- Comparing objects:
  - int, float, double etc
    - <, > are defined naturally based on values

ArrayList<Integer>

- Student
  - int id
  - String name
  - int age

```
Student s1 = new ...
Student s2 = new ...
s1 < s2; // Compile error
```

- Comparable interface
  - public interface Comparable<T>{
    public int compareTo(T Other);
    }

Arrays.sort (Comparable[] array)

comparing based on IDs

```
Student s1 = new Student(101, "A", 18);
Student s2 = new Student(102, "C", 17);
Student s3 = new Student(103, "B", 20);
```

$s1 < s2 \iff s1.compareTo(s2) < 0$

$$\begin{cases} a.compareTo(b) < 0 \Rightarrow a < b \\ a.compareTo(b) > 0 \Rightarrow a > b \\ a.compareTo(b) == 0 \Rightarrow a == b \end{cases}$$

we should return true for a.equals(b)

- public class Student implements Comparable<Student>{

```
    int id;
    String name;
    int age;
    public int compareTo ( Student that ) {
        if (this.name.equals(that.name)) {
            if (this.age < that.age) return -1;
            if (this.age > that.age) return +1;
            return 0;
        return this.age - that.age;
        }
        return this.name.compareTo(that.name);
    }
}
```

Let's say in our app, the natural way to compare students is by name (then age if name is same)

- Comparator interface

```
public interface Comparator<T> {
    public int compare(T o1, T o2);
}
```

$o1 < o2 \iff c.compare(o1, o2) < 0$

$o1 > o2 \iff c.compare(o1, o2) > 0$

$o1 == o2 \iff c.compare(o1, o2) == 0$

```
Student[] students = {...};
```

```
public class Student implements Comparable<Student> {
    public static final Comparator<Student> BY_ID = new IDComparator<>();
    public static final Comparator<Student> BY_ID_REV = new ReverseIDComparator<>();
    private static class IDComparator
            implements Comparator<Student> {
        int compare(Student s1, Student s2) {
            return s1.id - s2.id;
        }
    }
    private static class ReverseIDcomparator
            implements Comparator<Student>{
        public int compare(Student s1, Student s2) {
            //return s2.id - s1.id;
            return -(s1.id - s2.id);
        }
    }
    public int compareTo(Student that){
        if (this.name.equals(that.name)) return this.age - that.age;
        return this.name.compareTo(that.age);
    }
}
```

```
Arrays.sort(students);
```

sorts student objects by name (& then age in case of ties)

```
Arrays.sort(students, Student.BY_ID);
```

sort students by ID in ascending order

```
Arrays.sort(students, Student.BY_ID_REV);
```

sort students by ID in descending order.